

Computer Programming

Unit 2: Scanners

Contents

2.1 The Role of the Scanner

2.2 Recognizing Words

2.2.1 A Formalism of Recognizer

2.2.2 Recognizing Complex Words

2.3 Regular Expression

2.3.1 Formalizing the Notation

2.3.2 Closure properties of REs

2.4 From Regular Expression to Scanner

2.4.1 Nondeterministic Finite Automata

2.4.2 Regular Expression to NFA: Thomson's Construction

2.4.3 NFA to DFA

2.4.4 Using a DFA as a Recognizer

From Regular Expression to Scanner

- I. Nondeterministic Finite Automata
- II. Regular Expression to NFA: Thomson's Construction
- III. NFA to DFA
- IV. Using a DFA as a Recognizer

Finite automata

finite – limited

automata - self-acting, self-working

- *is a **state machine** that takes a **string of symbols as input** and **changes** its **state** accordingly.*

i.e. recognize token, patterns in input sequences

- **Thus**, it is a **recognizer** for **regular expressions**.

- An automaton with a **finite number of states** is called a **Finite Automaton (FA) or Finite State Machine (FSM)**.

A finite automaton can be can be represented by a 5-tuple

$$FA = (Q, \Sigma, \delta, q_0, F)$$

where,

Q = Finite set of states

Σ = Set of input symbol or input alphabet

δ = Transition function

q_0 = Initial state ($q_0 \in Q$)

F = Final states ($F \subseteq Q$)

Σ - Sigma, δ - delta

$F \subseteq Q$ - F is a subset of Q (i.e., every element of F is also in Q)

Finite Automaton can be classified into two types:

1. Deterministic Finite Automaton (DFA)
2. Non-deterministic Finite Automaton (NDFA / NFA)

Deterministic Finite Automaton (DFA)

- In DFA, for a **input symbol**,
the **machine can move to only one possible state** in the machine.

i.e. there is a **single unique state** or only one transition.
- FA is **deterministic** as given the **current state** we know what **next state** will be.

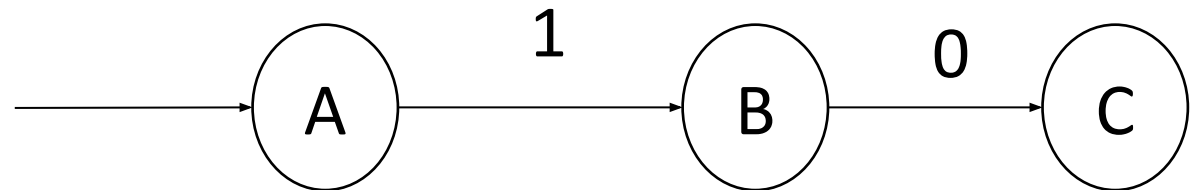
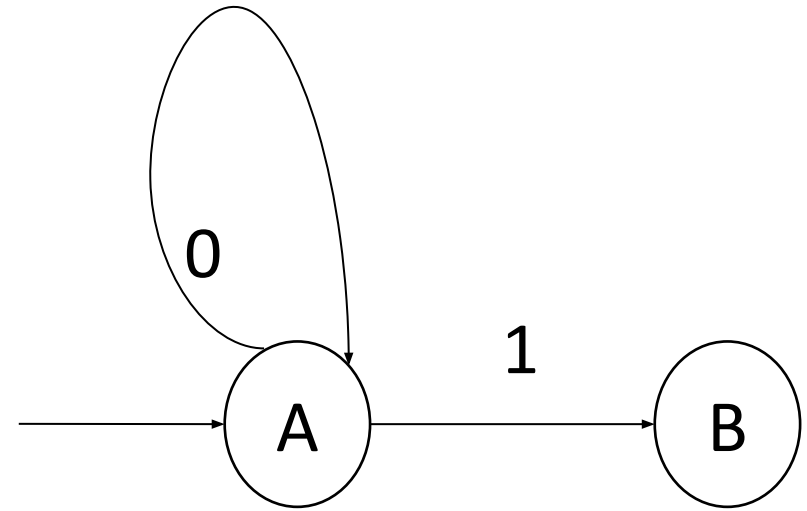
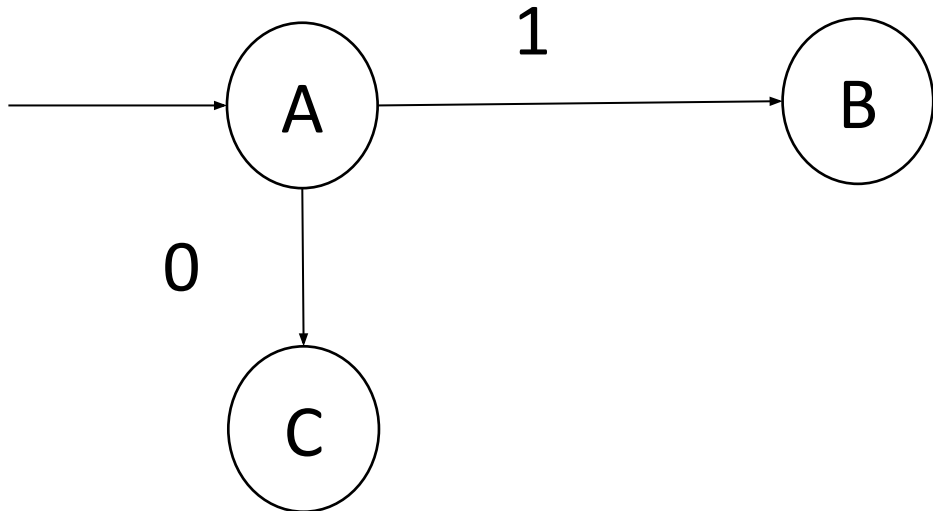
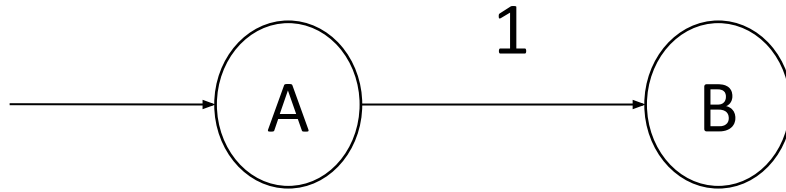
i.e. the **exact state** to which the **machine moves can be determined**

Hence called Deterministic Automaton

summary

- given the **current state** we know what **next state** will be.
- It has no choices or randomness.
- Simple and easy to design

Eg,



An DFA is a 5-tuple
 $(Q, \Sigma, \delta, q_0, F)$

where,

Q = non-empty finite set of states ($q_0, q_1, q_2, \dots$).

Σ = non-empty finite set of input symbols / input alphabet

δ = transition function

that maps a current state (Q) and an input symbol (Σ) to a state (Q)

i.e. $\delta : Q \times \Sigma \rightarrow Q$

q_0 = non-empty starting or initial state (*single state not set of states*)
($q_0 \in Q$)

F = set of final or accepting state ($F \subseteq Q$)
(can be empty or non-empty)

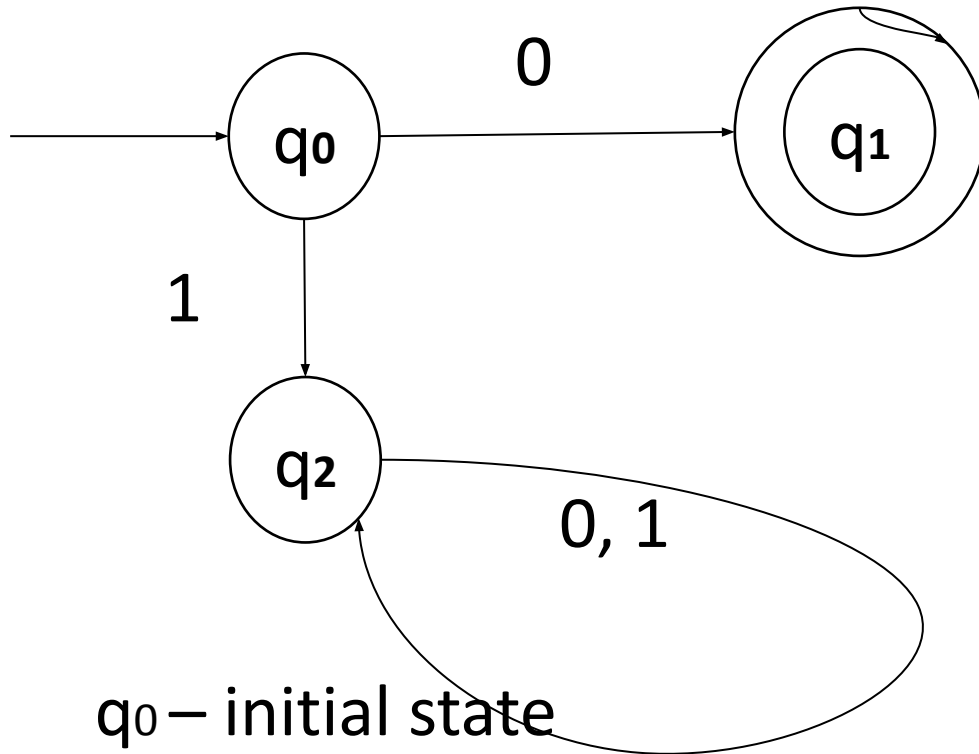
- DFA does **not allow** any **null transitions**,
meaning every **state** must have a **transition defined** for every **input symbol**.

i.e. Epsilon move is not allowed in DFA

Q1. Design a DFA that accepts all the strings that **start with 0**.

$L = \{0, 00, 01, 000, 010, 011, 0000, \dots\}$

0, 1



q_0 – initial state

q_1 – final state

q_2 – dead/trap state

Required DFA is given by

$DFA = (Q, \Sigma, \delta, q_0, F)$

where,

$Q = \{q_0, q_1\}$

$\Sigma = \{0, 1\}$

δ is given by figure

$q_0 = q_0$ (no curly bracket because q_0 is a single state, not a set)

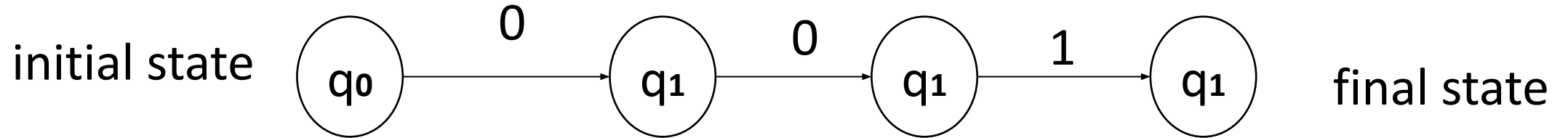
$F = \{q_1\}$

Transition Table

State	0	1
$\rightarrow q_0$	q_1	q_2
$*q_1$	q_1	q_1
q_2	q_2	q_2

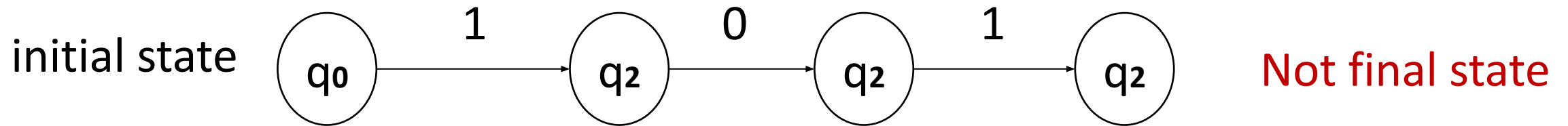
Trace:

check string 001 is accepted in the DFA design we made

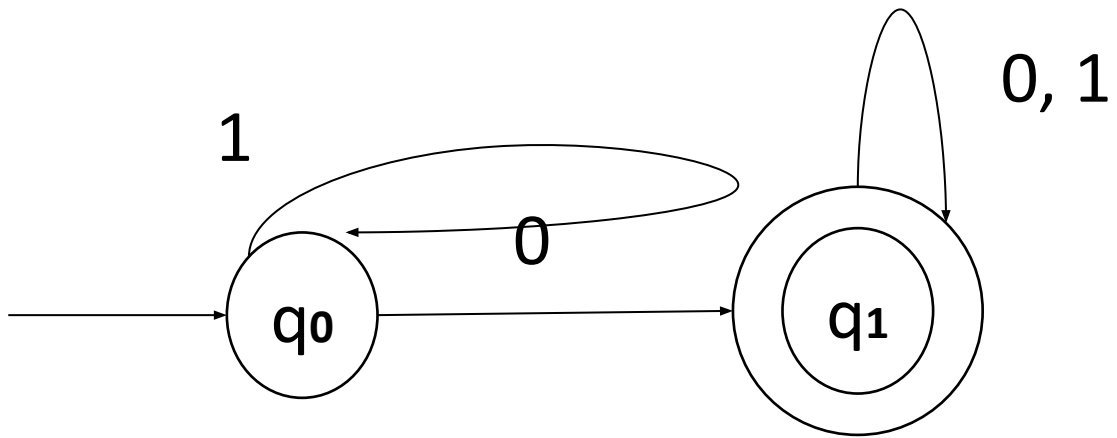


001 is accepted

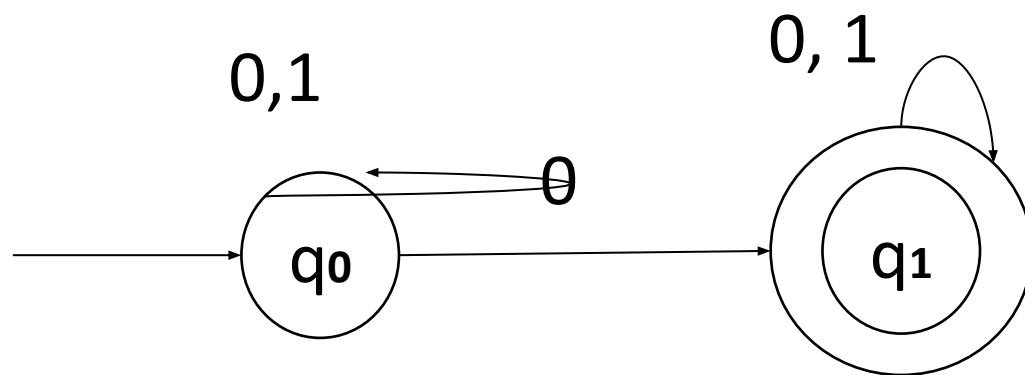
check string 101 is accepted in the DFA design we made



101 is not accepted



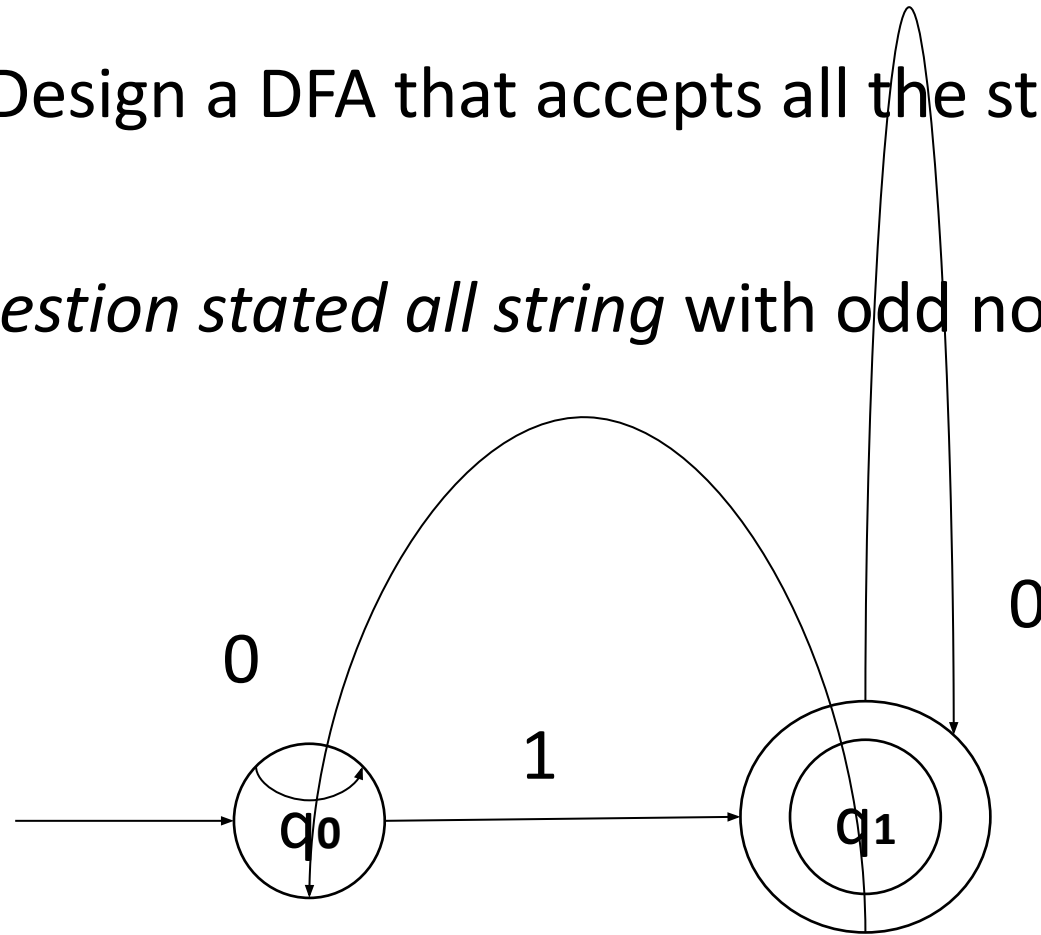
wrong as question clearly states
all the strings that **start with 0**.



wrong

2. Design a DFA that accepts all the strings that **with odd no. of 1's**

*Question stated all string with odd no. of 1's **not starting with 1***



$L = \{ 1, 01, 10, 001, 010, 100, 111, \dots \}$

Accepts odd no. of 1's

- 1, 111, 11111, 1111111
- 01, 10
- 001, 010, 100, 111

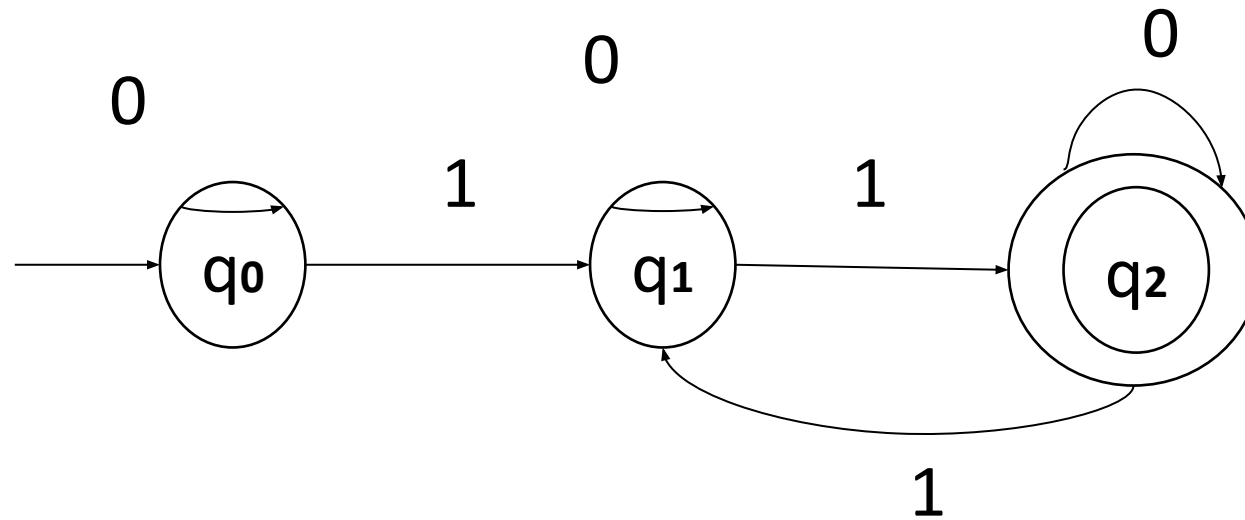
Not accepted

- 0 – **not final state**
- 00, 11,
- 110011 – **not final state**

3. Design a DFA that accepts all the strings **with even no. of 1's**

$$\Sigma = \{0,1\}$$

$$L = \{11, 1111, 111111, \dots 011, 0101, 010111, \dots\}$$



4. Design a DFA that accepts all sets of strings over $\{0, 1\}$ of length 2

$\Sigma = \{0,1\}$

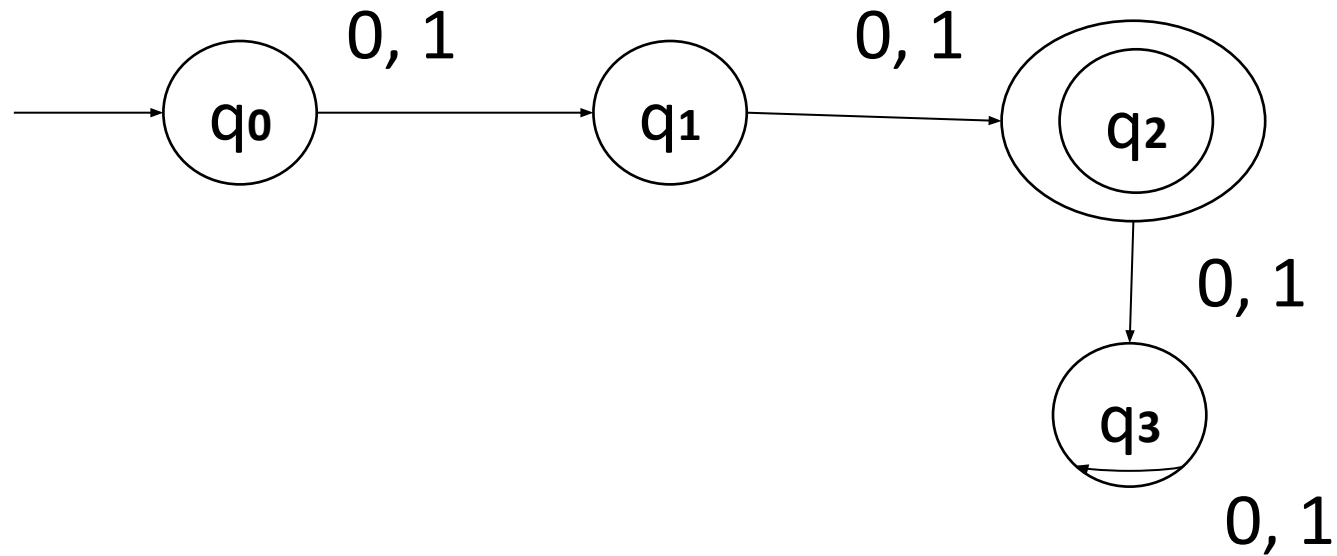
string of length 1 = 0,1

string of length 2 = 00, 01, 10, 11

string of length 3 = 000, 001, 010, 011, 100, 101, 110, 111

....

$L = \{00, 01, 10, 11\}$



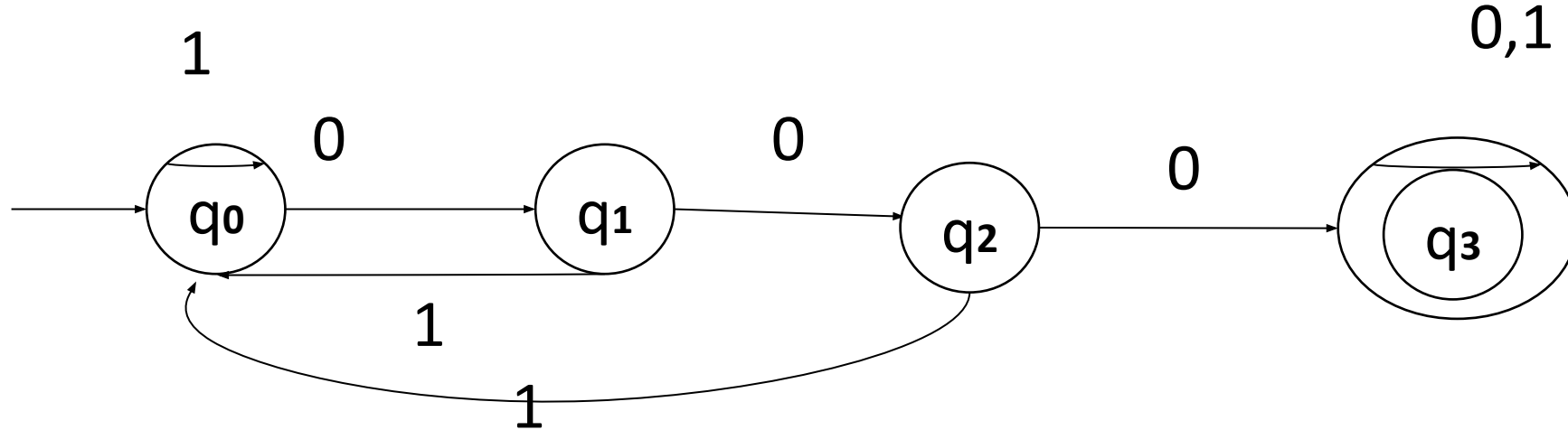
Eg, 00, 01, 10, 11 - accepted

0, 1 – 0 or 1

q3 – dead/trap
state

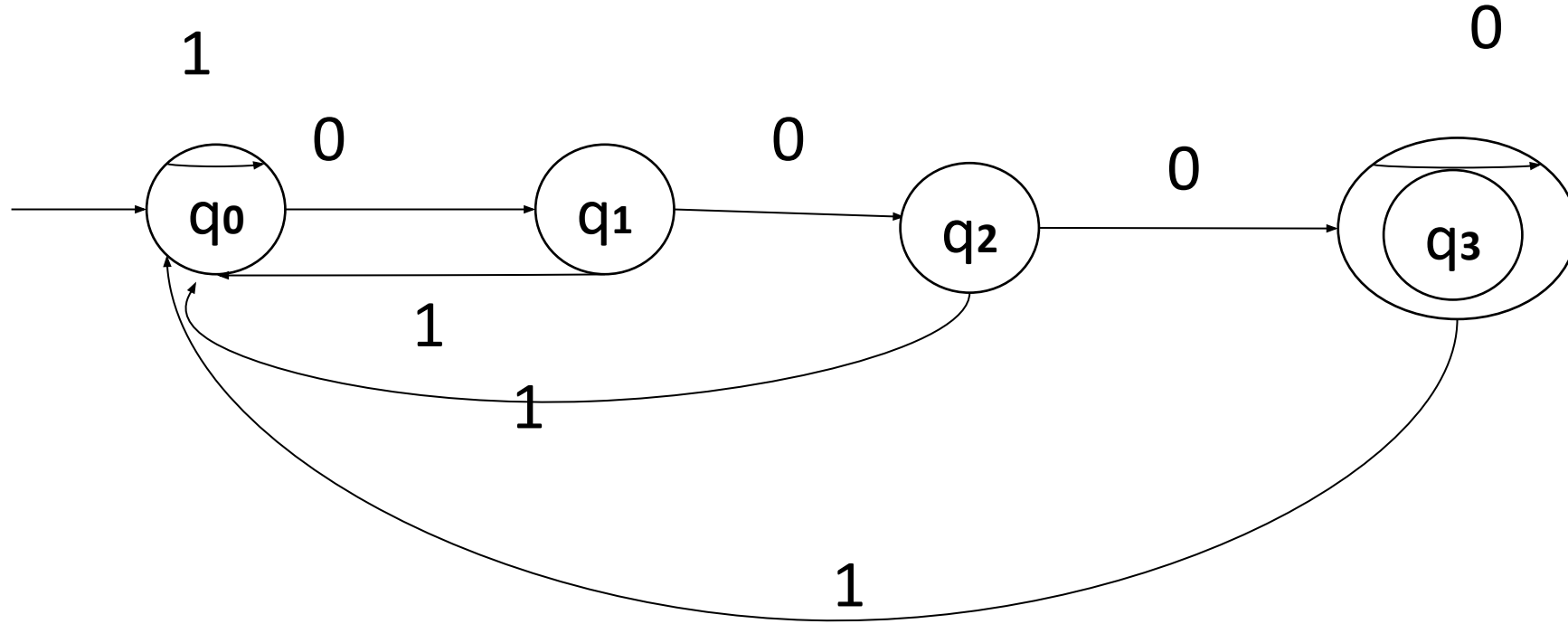
5. Design a DFA that accepts all sets of strings over $\{0, 1\}$ **that contain three consecutive 0's**

$L = \{000, 0001, 1000, 10001, 101000, 1001000, \dots\}$



6. Design a DFA that accepts all sets of strings over $\{0, 1\}$ ending with three consecutive 0's

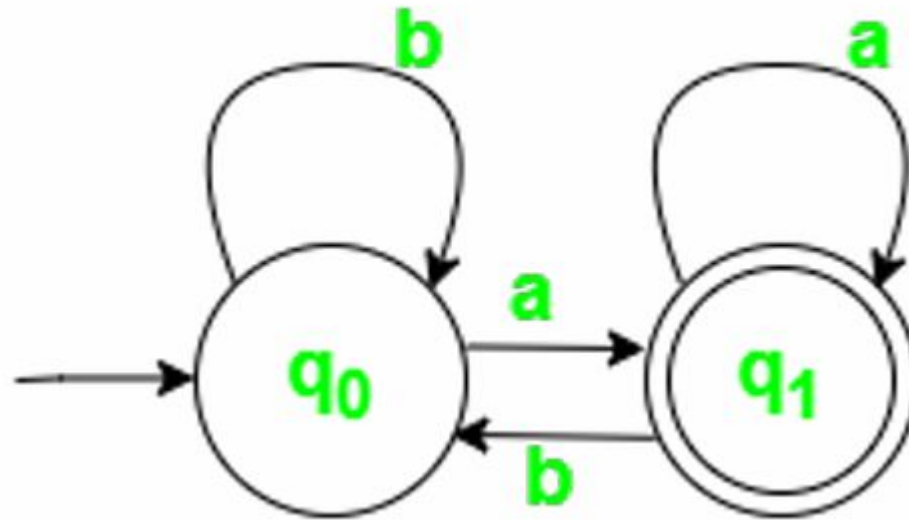
$L = \{000, 1000, 1000000, \dots\}$



7. Construct a DFA that accepts all strings **ending with 'a'**

$\Sigma = \{a,b\}$

$L = \{a, ba, baa, baba, babbaa, \dots\}$

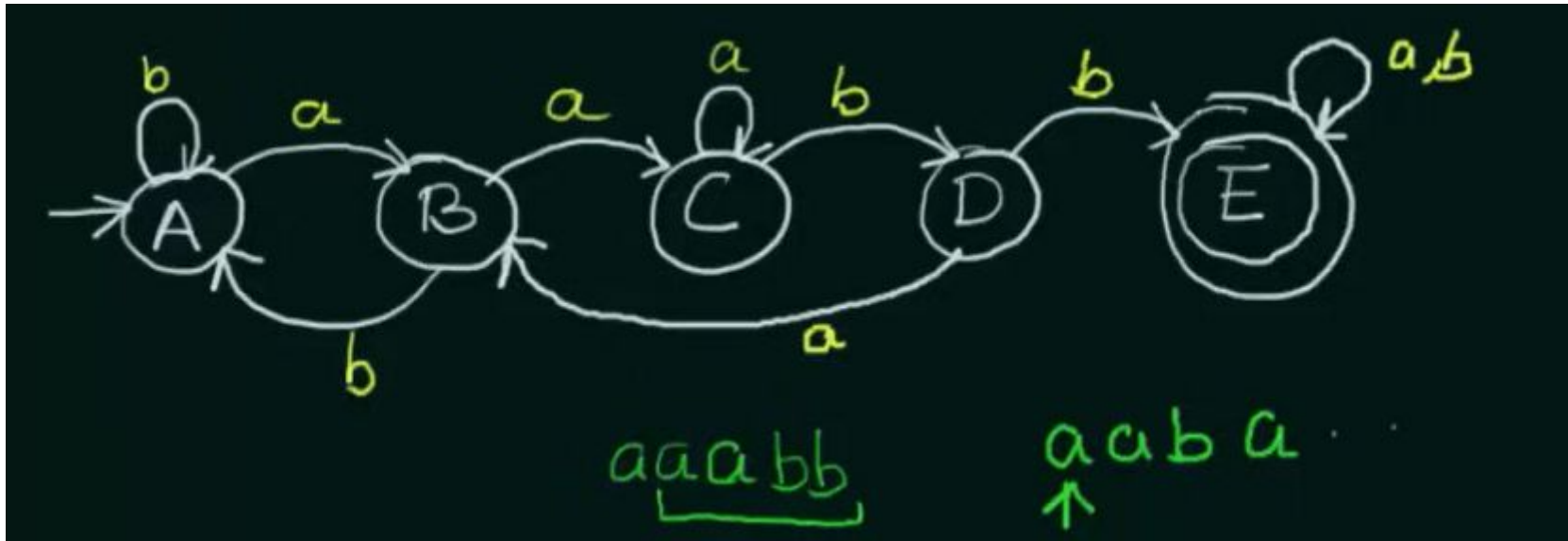


8. Construct a DFA that accepts any string over $\{a,b\}$ that **doesnot contain aabb** in it.

$$\Sigma = \{a, b\}$$

□ First we have to construct a DFA that accepts any string over $\{a,b\}$ that **contain aabb** in it

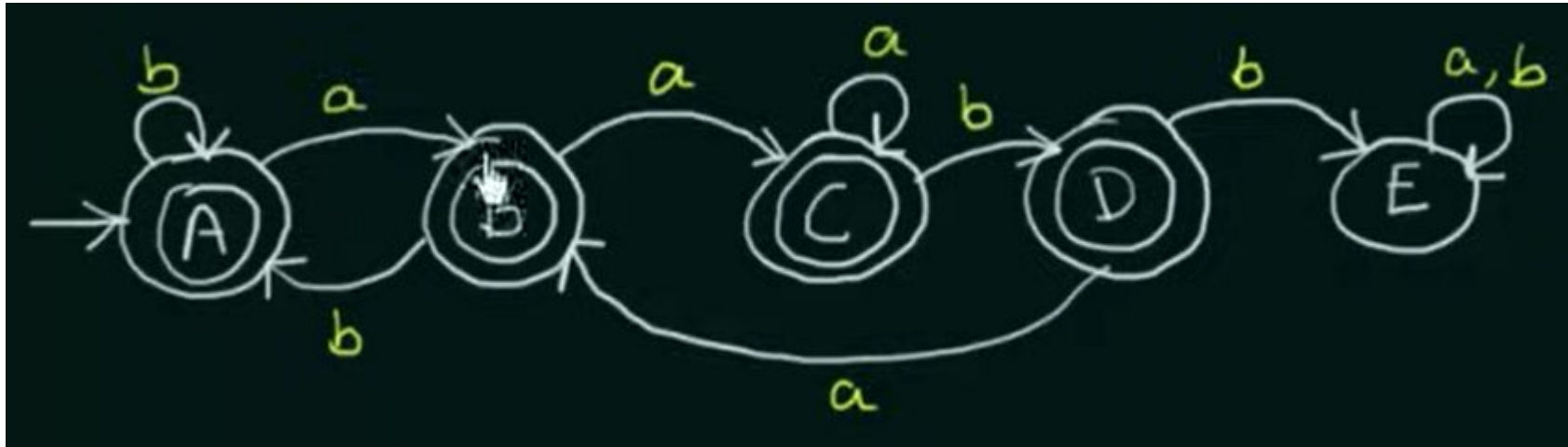
*Question stated string over $\{a,b\}$ that **contain aabb** in it **not starting with aabb or ending with aabb***



Here is a DFA that accepts any string over $\{a,b\}$ that **contain aabb** in it.

Now to achieve DFA that **doesnot contain aabb in it**, we have to

- Flip the states
- Make the final state into non-final state
- Make the non-final state into final state



aabb will not be accepted and all other strings will be accepted

Non-deterministic Finite Automaton (NDFA / NFA)

- In NFA, for a **particular input symbol**,
the machine **may move to**
one state
or **many states**
or **no state ($\emptyset$)** in the machine.

$\emptyset$ (phi / empty set)
- No string at all
- Empty language

- NFA is **non deterministic**, as given the **current state** there could be **multiple next state**

i.e. the **exact state** to which the machine moves cannot be determined.

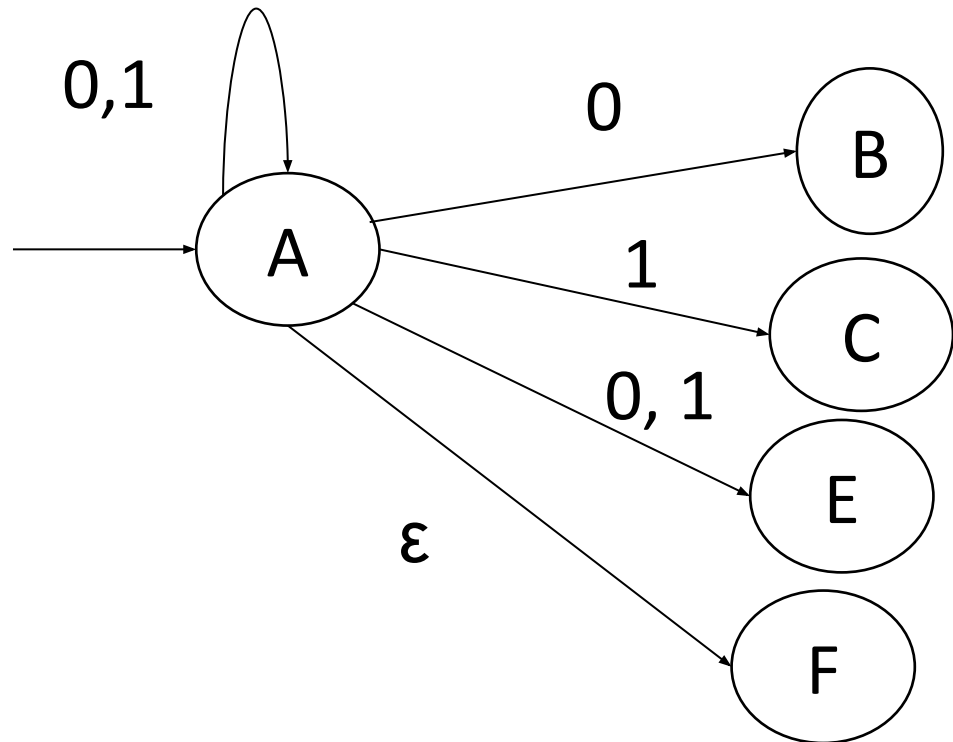
Hence, it is called Non-deterministic Automaton.

- It is **slower recognizer** but it make take **less spaces**.

Summary

- given the **current state** there could be **multiple next state**
- The next state can be chosen at random
- All next state may be chosen in parallel

Eg,



(epsilon) ϵ means: “move without consuming input”

Thus NFA **allow null transitions**,
But DFA does **not allow** any **null transitions**

An NFA is a 5-tuple

$$(Q, \Sigma, \delta, q_0, F)$$

where,

Q = non-empty finite set of states ($q_0, q_1, q_2, \dots$).

Σ = non-empty finite set of input symbols / input alphabet

δ = transition function

that maps a current state (Q) and an input symbol (Σ) to a
set of possible next states

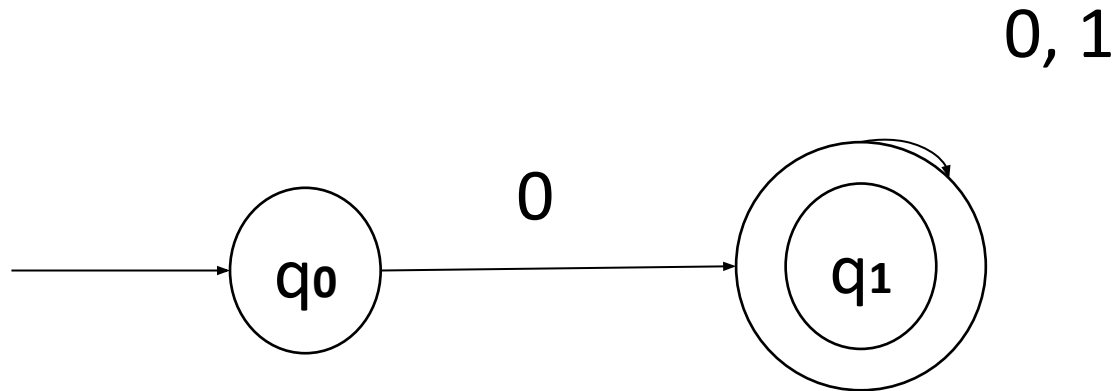
$$\delta : Q \times \Sigma \rightarrow 2^Q$$

q_0 = starting or initial state ($q_0 \in Q$)

F = final or accepting state ($F \subseteq Q$)

Q1. Design a NFA that accepts set of all the strings that **start with 0**.

$L = \{0, 00, 01, 000, 010, 011, 0000, \dots\}$



q_0 – initial state

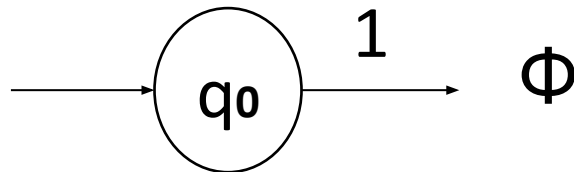
q_1 – final state

Eg, 001

$\square q_0 \square q_1^0 \square q_1^0 \square q_1^1$

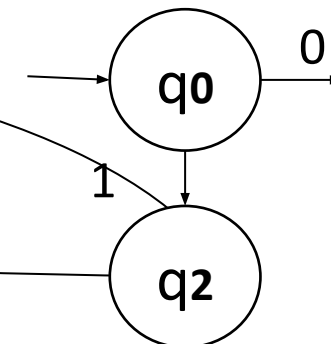
What happens when q_0 gets **input 1**

Eg, 101



State q_0 goes to no state i.e. doesn't have anywhere to go so it is known as **Dead configuration**

In DFA, on input 1, it went to dead state



Required NFA is given by

$$\text{NFA} = (Q, \Sigma, \delta, q_0, F)$$

where,

$$Q = \{q_0, q_1\}$$

$$\Sigma = \{0, 1\}$$

δ is given by figure

$$q_0 = q_0$$

$$F = \{q_1\}$$

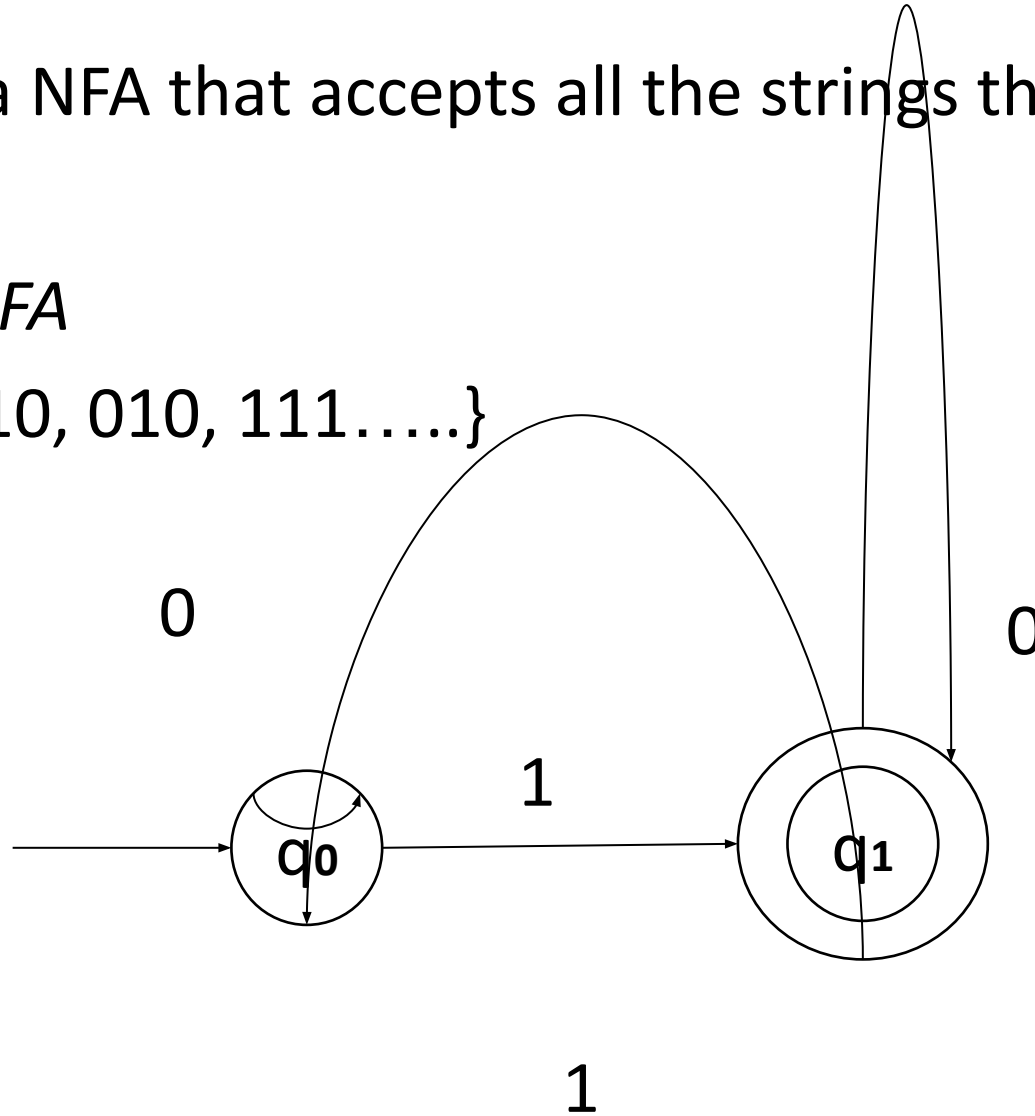
Transition Table

State	0	1
$\rightarrow q_0$	q_1	$\emptyset$
$*q_1$	q_1	q_1

2. Design a NFA that accepts all the strings that **with odd no. of 1's**

Same as DFA

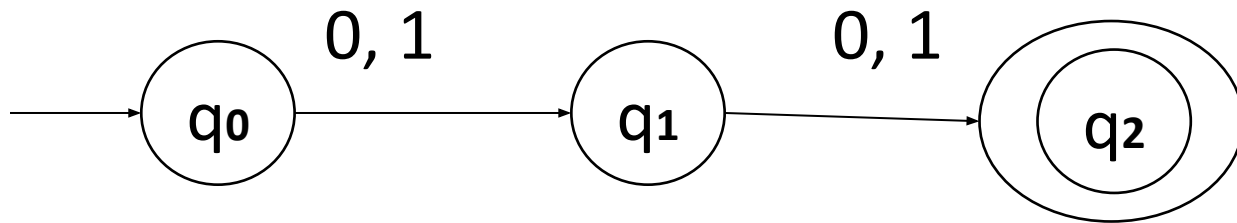
$L = \{1, 01, 10, 010, 111, \dots\}$



4. Design a NFA that accepts all sets of strings over $\{0, 1\}$ of length 2

$$\Sigma = \{0, 1\}$$

$$L = \{00, 01, 10, 11\}$$



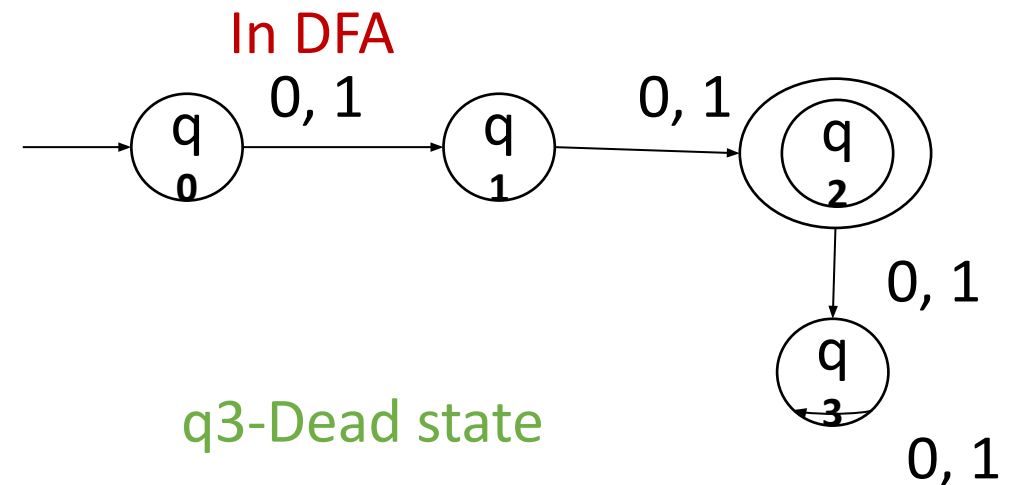
$0, 1 - 0 \text{ or } 1$

Eg, 00, 01, 10, 11 – accepted

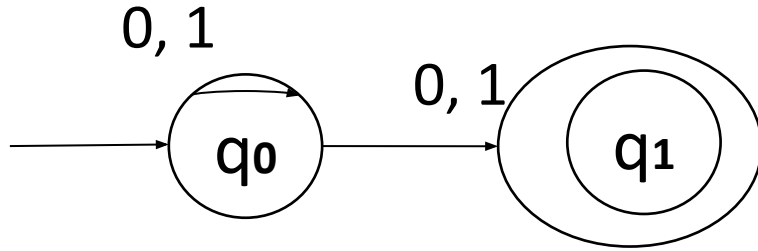
Eg, 001

$q_0 \xrightarrow{0} q_1 \xrightarrow{0} q_2 \xrightarrow{1} \Phi$

Rejected



Why not



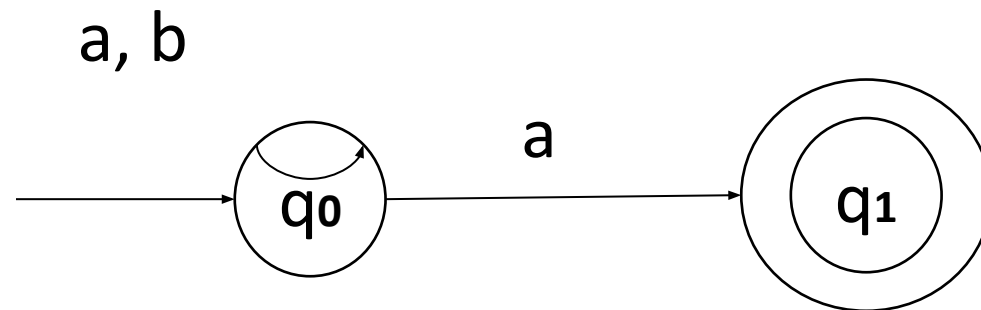
- For length = 2 → we need **3 states**
- But we only have **two states**: q0 (start) and q1 (final).
- To accept all strings over {0,1} of **length 2**, the automaton must:
 - Count **exactly 2 input symbols (0 or 1)**
 - **Accept only after 2 inputs**
 - And Reject 0,1, 001, 101, ...
- *but This also accepts 001, 101*

001- 0 goes to q0, 0 again goes to q0, 1 goes to q1

7. Construct a NFA that accepts all strings **ending with 'a'**

$\Sigma = \{a,b\}$

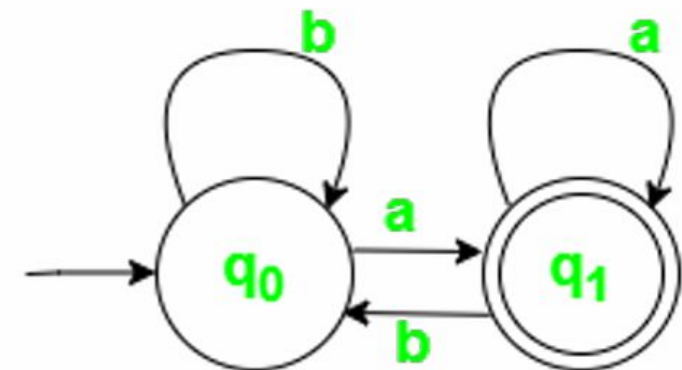
$L = \{a, aa, ba, baa, baba, babbaa, \dots\}$



In NFA, q_1 doesn't have to go anywhere.

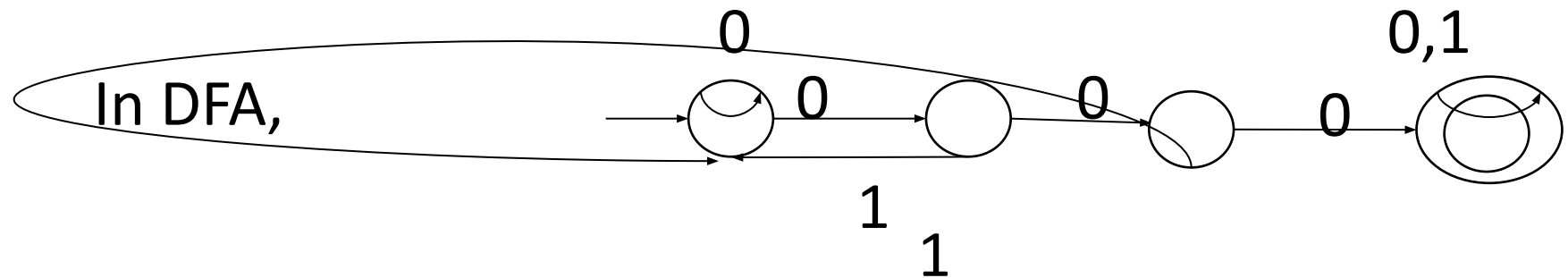
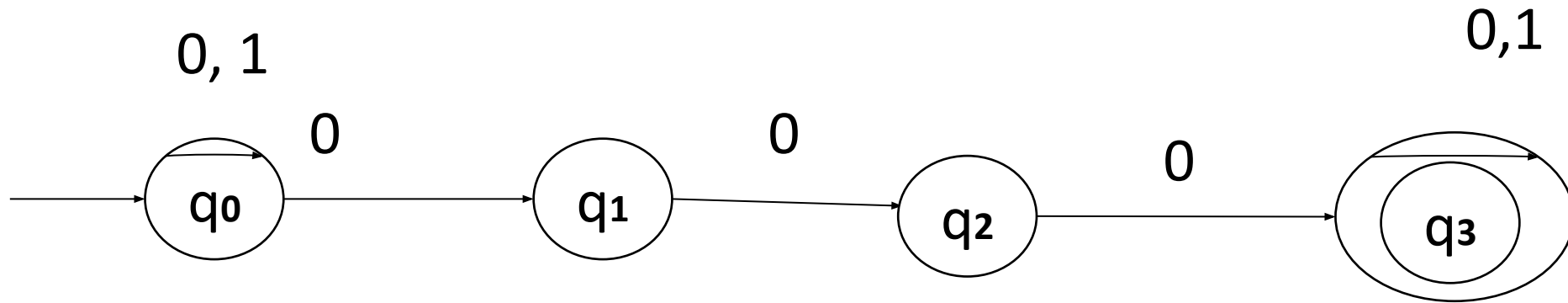
If more input comes after q_1 , that path dies (so only paths that end exactly on 'a' are accepted).

In DFA



5. Design a NFA that accepts all sets of strings over $\{0, 1\}$ that contain three consecutive 0's

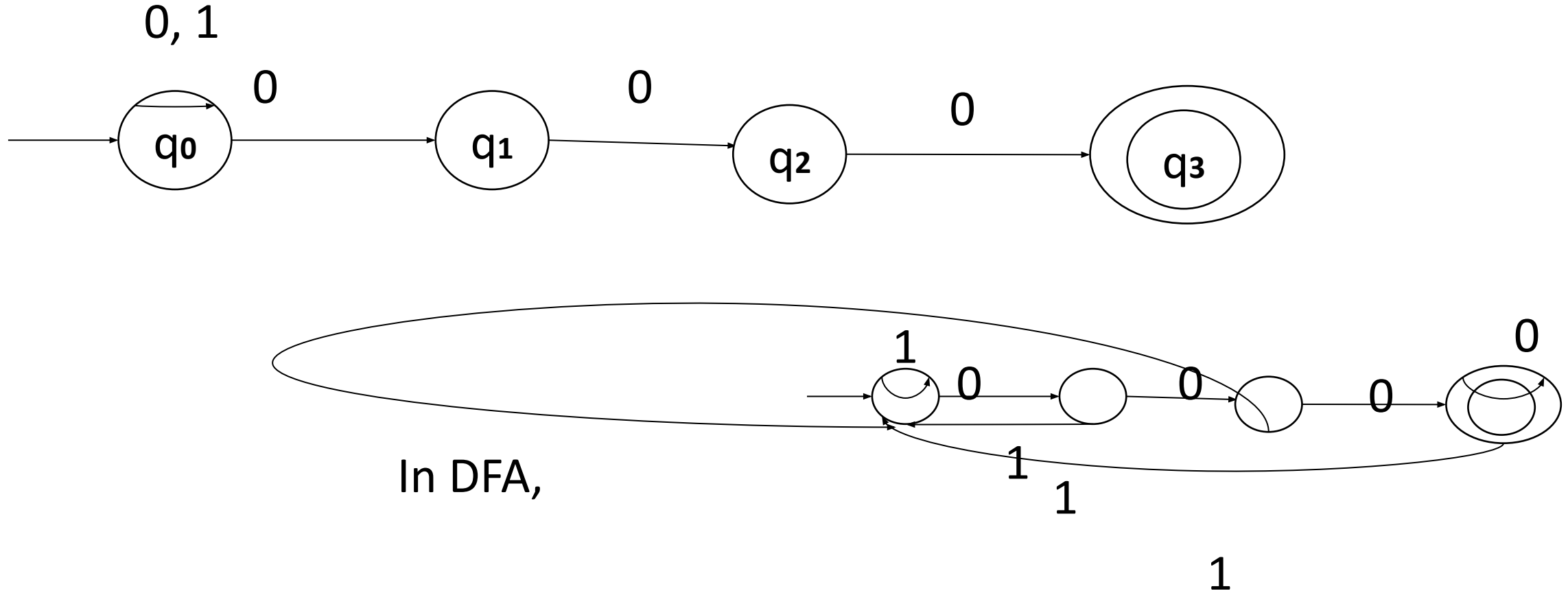
$$L = \{000, 0001, 1000, 10001, \dots\}$$



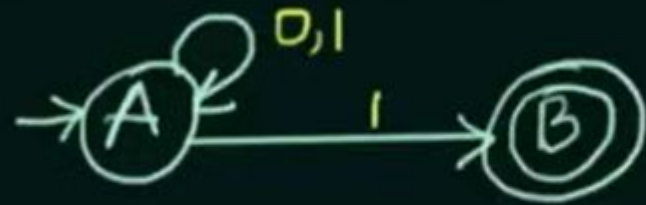
- q_1 on input 1 $\rightarrow \emptyset$
- q_2 on input 1 $\rightarrow \emptyset$

6. Design a NFA that accepts all sets of strings over $\{0, 1\}$ **ending with three consecutive 0's**

$L = \{000, 1000, 1000000, \dots\}$

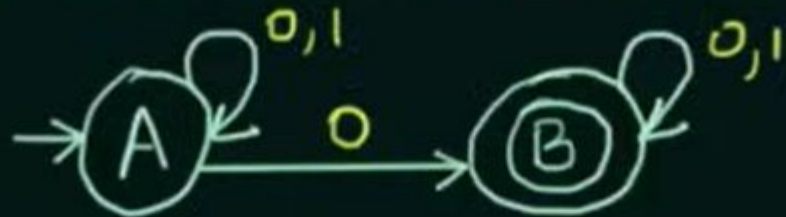


Ex 1) $L1 = \{ \text{Set of all strings that ends with '1'} \}$

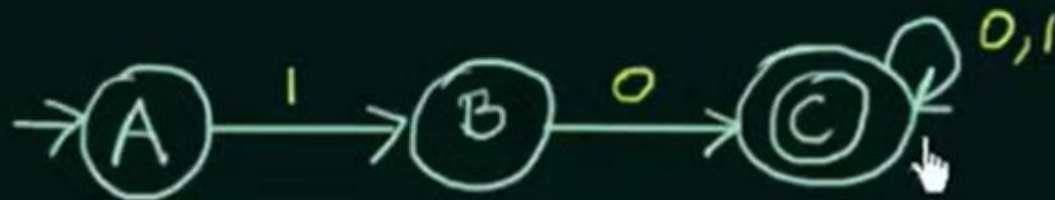


01, 001, 0001, 0^*1 , 1,
101, 1101,

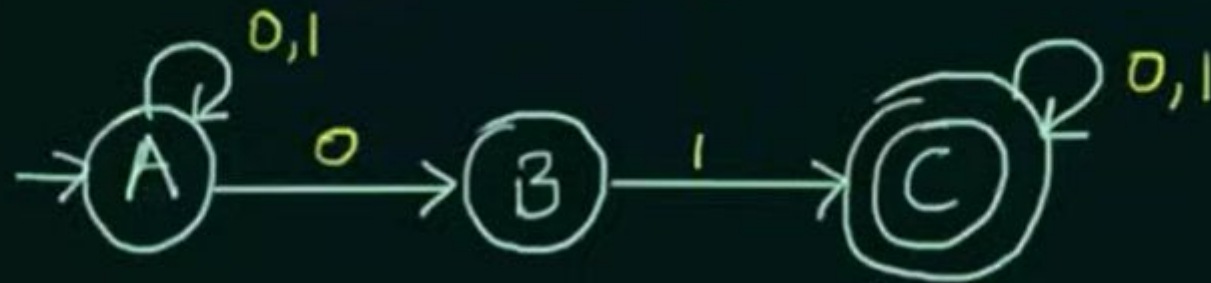
Ex 2) $L2 = \{ \text{Set of all strings that contain '0'} \}$



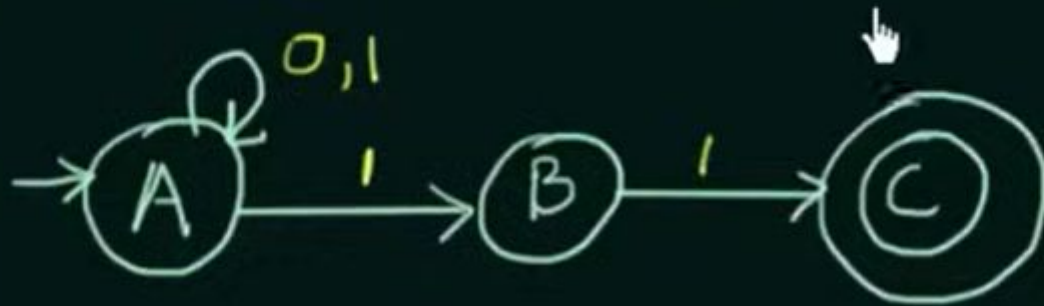
Ex 3) $L3 = \{ \text{Set of all strings that starts with '10'} \}$



Ex 4) $L4 = \{ \text{Set of all strings that contain '01'} \}$



Ex 5) $L5 = \{ \text{Set of all strings that ends with '11'} \}$



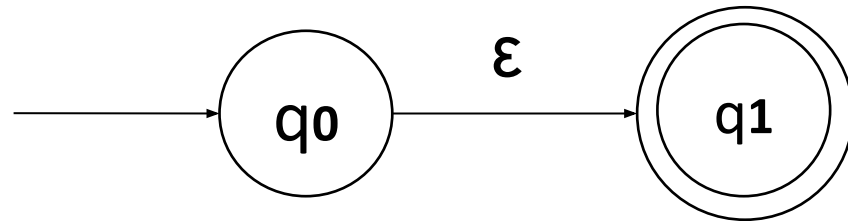
Difference

- DFA is a special case of NFA.
- There is only **difference** between NFA and DFA is in the **transition function**.
- In **NFA** transition from **one state to multiple states** take place

while in **DFA** transition from **one state to only one possible next state** take place.

ϵ - NFA

- In NFA, if a **transition made without any input symbol** is called **ϵ -NFA**.
- Here we need ϵ -NFA because the regular expressions are easily convertible to ϵ -NFA.



- can process the **empty string** ϵ as an input and can move from one state to another **without consuming input symbol using ϵ -transitions.**